

# 剖析主题：EasyDistillation 蒸馏法格点 QCD 核心部分穷尽 剖析（代码-物理交叉向）

opencode pure

2026 年 8 月 17 日

## 摘要

本报告对 **EasyDistillation**（格点 QCD 蒸馏法软件包）执行 pure 穷尽剖析。项目性质判定为 **代码-物理交叉向**：核心层（`generator/`、`quark_contract.py`、`quark_diagram.py`、`symmetry/`、`insertion/`）为物理计算代码，每个关键符号都对应蒸馏法中的物理对象，独特性载体正是“代码符号 ↔ 物理对象”的双向对应。穷尽枚举核心部分候选 14 项（C1–C14），三态分配：**深挖 5 / 浅析 8 / 排除 1**。五个深挖部分为：C1 自动 Wick 收缩、C2 拉普拉斯本征向量生成器（含 stout CUDA 核）、C3 ElementalGenerator、C4 强子不可约表示与小群投影、C5 PerambulatorGenerator，每部分均按 15 步全流程重现并给出代码-物理双向映射表。最强竞争力结论：以**邻接矩阵编码 Wick 收缩图并自动转 einsum**的收缩引擎，配合群论自动枚举强子算符，是“手写收缩脚本”朴素实现难以企及的核心创新。实测：两个纯逻辑测试套件（19+8 例）本机通过；PyQUDA/GPU 数据路径未在本机验证。

## 目录

## 1 项目定位与核心部分清单

### 1.1 项目性质判定

判定依据（实测统计）：`lattice/` 41 个 `.py` 全部为物理计算代码（张量收缩、群论、反演），文档 `README.md/TODO.md/doc/README.md` 描述蒸馏法物理流程与数据形状，代码注释含大量物理量（`perambulator.py`:16-49 的参数说明、`doc/README.md`:3-65 的形状约定）。独特性载体为“符号 ↔ 对象”对应而非纯算法复杂度或纯推导链，故判定为**代码-物理交叉向**。

#### 要点

**项目性质：**代码-物理交叉向（依据：物理计算代码占比 ≈100%、物理对象贯穿全部核心模块、独特载体 = 代码符号 ↔ 物理对象双向映射）

### 1.2 核心部分总清单（穷尽枚举）

| 编号  | 核心部分                                                       | 处理态 | 独特性概述                                                   |
|-----|------------------------------------------------------------|-----|---------------------------------------------------------|
| C1  | 自动 Wick 收缩 (quark_contract.py + quark_diagram.py)          | 深挖  | 味结构符号代数 $\rightarrow$ 邻接矩阵 $\rightarrow$ einsum 收缩, 全自动 |
| C2  | 拉普拉斯本征向量生成器 (eigenvector.py + stout_smear.cu)              | 深挖  | scipy/cupyx/QUDA 三路径 + stout 平滑 CUDA 核                  |
| C3  | ElementalGenerator (elemental.py)                          | 深挖  | 协变导数乘积规则展开 + 动量平面波投影                                    |
| C4  | 强子不可约表示与小群投影 (hadron*.py + symmetry/ + insertion/)         | 深挖  | $O_h^D$ / 小群不可约表示自动枚举算符                                 |
| C5  | PerambulatorGenerator (perambulator.py)                    | 深挖  | PyQUDA 反演 + $\gamma_5$ -厄米性 + MPI 片切分                   |
| C6  | Diagram 化简与批量计算引擎 (quark_diagram.py simplify/calc_diagram) | 浅析  | 图同构去重、连通分量拆分, 支撑大规模收缩                                   |
| C7  | MPI 任务分发与原子写 (dispatch.py)                                 | 浅析  | 文件队列 + 原子锁的组态批处理                                        |
| C8  | 惰性内存映射 IO (filedata/)                                      | 浅析  | mmap 按需切片 + 吞吐统计                                        |
| C9  | 双粒子 JPC 算符 (symmetry/two_particle.py)                      | 浅析  | 球谐 + CG 系数构造 $J^{PC}$ 双粒子基                              |
| C10 | 费曼图绘制 (quark_draw.py)                                      | 浅析  | 邻接矩阵 $\rightarrow$ 可视化                                  |
| C11 | 噪声向量/广义与密度 perambulator (noisevector.py 等)                 | 浅析  | 随机蒸馏与高模、 $\gamma$ -动量扩展反演                               |
| C12 | 后端抽象与数据预设 (backend.py, preset.py, data.py)                 | 浅析  | numpy/cupy 切换 + 四类数据形状预设                                |
| C13 | 物理常量与动量字典 (constant.py, mom_dict.py)                       | 排除  | 纯查表常量, 无独特性                                             |
| C14 | 工程配置 (README/TODO/LICENSE/requirements/.git*)              | 排除  | 非核心代码                                                   |

表 1: 核心部分总清单 (三态填满, 数据来源: 全库穷尽枚举实测)

## 2 核心部分深度剖析

### 2.1 C1 自动 Wick 收缩 (quark\_contract.py + quark\_diagram.py)

#### 代码-物理映射

**映射原则:** 每个关键代码符号必有物理对象, 双向可查, 无孤儿符号。

| 代码符号                     | 物理对象                              | 物理含义与参考源                                                                                |
|--------------------------|-----------------------------------|-----------------------------------------------------------------------------------------|
| Qurak(flavor, tag, anti) | 场算符 $\psi_f(x) / \bar{\psi}_f(x)$ | 带味的裸夸克符号 (tag 携带强子序号与时间); quark_contract.py:17-36                                       |
| Tag(tag, time)           | 时空/粒子标号                           | tag//3= 强子号、tag%3= 强子内夸克号、time= 时间片; quark_contract.py:12-14                            |
| Propagator               | 夸克传播子 $S^f$                       | 符号 $S^f$ (汇, 源); 同位标为 local; quark_contract.py:39-61                                    |
| HadronFlavorStructure    | 强子味结构                             | 介子 (2 字)/重子 (3 字)/反重子 (bar{}), baryon_num $\in \{0, 1, -1\}$ ; quark_contract.py:64-107 |
| _quark_contract          | Wick 收缩                           | 反夸克锚定 + 同位素夸克递归配对 + 交换符号; quark_contract.py:216-245                                     |
| adjacency_matrix         | Wick 收缩图                          | 顶点 = 强子、矩阵元 = 传播子编号 (重子为 $3 \times 3$ 子矩阵); quark_contract.py:179-208                   |
| QuarkDiagram.analyse     | 图 $\rightarrow$ 张量网               | BFS 分解连通分量, 分配自旋/色下标生成 einsum 式; quark_diagram.py:15-79                                 |
| compute_diagrams         | 关联函数数值                            | 各图逐项 einsum 求值; quark_diagram.py:261-273                                                |

表 2: C1 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** Wick 收缩是把强子关联函数  $\langle O_1 O_2 \dots \rangle$  按费曼拓扑展开为传播子乘积组合的运算, 手写极易出错 (交换符号、味匹配、简并)。该库把它抽象为“符号代数 + 图论”两层。前置条件: 仅依赖 sympy 与 numpy (requirements.txt:1-4)。

**A2 输入.** 表达式 (HadronFlavorStructure 的乘积)、粒子列表、简并开关 degenerate (quark\_contract.py:110-124)。

**A3 输出 (应是什么).** Diagram 的符号表达式 (系数 + 邻接矩阵 + 时标 + 传播子表), 如 meson-meson 收缩得  $\pm[[0, 1], [1, 0]]$  对角拓扑。

**A4 整体框架.** 两步: quark\_contract (味  $\rightarrow$  传播子符号与邻接矩阵, quark\_contract.py:125-213)  $\rightarrow$  QuarkDiagram (邻接矩阵  $\rightarrow$  einsum, quark\_diagram.py:15-79)  $\rightarrow$  compute\_diagrams (求值)。

**A5 关键细节.** 递归配对用“首个反夸克为锚” (:221-222); 简并 u/d 统一传播子味 "q" (:233-236); 交换符号  $(-1)^{i-j}$  (:225-232); BFS 遍历避免重复处理 (quark\_diagram.py:29-50)。

```

1 def _quark_contract(symbol_list, result_list, result, degenerate):
2     if symbol_list == []:
3         result_list.append(Mul(*result))
4         return
5     for i, src in enumerate(symbol_list):
6         if src.anti:
7             break
8     for j, snk in enumerate(symbol_list):
9         if not snk.anti and snk.flavor == src.flavor:
10            if i > j:
11                symbol_list.pop(i)
12                symbol_list.pop(j)
13                factor = S(-1) ** (i - j - 1)
14            else:
15                symbol_list.pop(j)
16                symbol_list.pop(i)
17                factor = S(-1) ** (j - i)
18            if degenerate and (snk.flavor == "u" or snk.flavor == "d"):
19                prop = Propagator("q", src.tag, snk.tag)
20            else:
21                prop = Propagator(src.flavor, src.tag, snk.tag)
22            result.append(factor * prop)
23            _quark_contract(symbol_list, result_list, result, degenerate)
24            result.pop()
25            if i > j:
26                symbol_list.insert(j, snk)
27                symbol_list.insert(i, src)
28            else:
29                symbol_list.insert(i, src)
30                symbol_list.insert(j, snk)

```

**A6 任务如何正确执行.** 调用 quark\_contract(expr, particles, degenerate) (test\_quark\_contract.py:159) 数值阶段把结果代入 compute\_diagrams。

**A7 实际执行情况.** 本机实测 test\_quark\_contract.py: Ran 19 tests, OK (记录于 .pure.\*.log)。

**A8 实际输出情况.** meson-meson ( $\bar{u}d \times \bar{d}u$ ) 收缩输出  $-\dots q$ , 邻接矩阵  $[[1, 0], [0, 1]]$ , 与 test\_quark\_contract.py:184 断言一致。

**A9 结果分析.** 19 例测试覆盖 tag 不可变性、简并、重子-反重子、传播子符号表示

(test\_quark\_contract.py:12-222), 符号层自治性验证通过。

**A10 综合分析.** 该引擎是蒸馏法四阶段之后的核心抽象：物理上 Wick 收缩的“所有可能配对”即符号代数的“所有递归分支”，配对顺序产生置换符号，映射逐符号成立（见映射表）。

**A11 结尾总结. 核心算法:** 递归 Wick 收缩  $\rightarrow$  邻接矩阵  $\rightarrow$  BFS 图分析  $\rightarrow$  einsum；正确性由 19 例符号测试保证。

**A12 结尾评价.** 优点：全自动、可符号化简；缺点：符号代数分支数随粒子数指数增长（未验证性能瓶颈），邻接矩阵仅存拓扑不存旋量组合细节。

**A13 补充说明.** 大规模（>4 顶点）收缩的耗时与内存未实测；数值路径（含 perambulator）未在本机运行。

**A14 参考源.** quark\_contract.py:110-245, quark\_diagram.py:15-79, 261-273, test\_quark\_contract.py:12-

**A15 附录.** 实测命令与输出见 .pure.2026-08-17-06-18-21.log。

## 2.2 C2 拉普拉斯本征向量生成器 (eigenvector.py + stout\_smear.cu)

### 代码-物理映射

**映射原则:** 每个关键代码符号必有物理对象，双向可查。

| 代码符号              | 物理对象                          | 物理含义与参考源                                                                      |
|-------------------|-------------------------------|-------------------------------------------------------------------------------|
| _Laplacian        | 协变拉普拉斯 $-\nabla^2$            | 对角 6、plaquette 链求和 (SA 求最小本征) ; eigenvector.py:11-26                          |
| backend.roll      | 位移算子 $\hat{T}_\mu$            | 相邻格点平移 (周期边界); eigenvector.py:19-24                                           |
| eigsh(which="SA") | 本征问题 $\nabla^2 v = \lambda v$ | 最低 $N_e$ 个本征解 (升序排序) ; eigenvector.py:246-251                                 |
| renorm_phase      | 本征向量相位                        | $v \rightarrow e^{-i \arg(v_1)} v$ 约定 (本征向量相位不确定) ; eigenvector.py:254-256    |
| project_SU3       | 幺正化                           | 迭代 $U \leftarrow \frac{1}{2}(U + U^{-\dagger})$ 到 SU(3); eigenvector.py:55-65 |
| stout_smear       | stout 平滑                      | 指数化 $U'_\mu = e^{iQ_\mu} U_\mu$ 降紫外噪声; eigenvector.py:215-232                 |
| stout_smear.cu    | GPU 平滑核                       | 逐链接 staple + 矩阵指数; stout_smear.cu:160-245                                     |
| laplacian_quda    | QUDA 本征求解                     | TR-Lanczos + 可选切比雪夫加速; eigenvector.py:259-312                                 |

表 3: C2 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** 蒸馏基底需要三维空间拉普拉斯的最低  $N_e$  本征向量；同时要平滑规范场以降低本征求解噪声。前置：规范场数据、numpy/scipy (CPU) 或 cupyx/QUDA (GPU)、PyQUDA (eigenvector.py:203-213, 259-262)。

**A2 输入.** 格点尺寸 latt\_size、规范场 GaugeField、 $N_e$ 、容差 tol (eigenvector.py:29-47)。

**A3 输出.** 每时间片 [Ne, Lz, Ly, Lx, Nc] 本征向量 + 本征值 (eigenvector.py:257)。

**A4 整体框架.** 三路径 (calc, :356-370) : cupy+QUDA  $\rightarrow$  laplacian\_quda; cupy/numpy  $\rightarrow$  laplacian\_cupy\_numpy; QUDA 拉普拉斯 + cupyx 求解  $\rightarrow$  laplacian\_quda\_scipy。平滑独立分发 (:215-232)。

**A5 关键细节.** 拉普拉斯用 `opt_einsum` 实现

$$(-\nabla^2 \psi)_a(x) = 6\psi_a(x) - \sum_{\mu=1}^3 [U_\mu(x)\psi(x + \hat{\mu}) + U_\mu^\dagger(x - \hat{\mu})\psi(x - \hat{\mu})]_a \quad (1)$$

即 (??); 对角项取 6 (`eigenvector.py:16-17`); 动量卷积用 `roll (:19-24)`。stout 平滑指数化  $e^{iQ}$  (`eigenvector.py:140-189` 与 CUDA 核 `stout_smear.cu:198-239` 逐点一致)。

**物理图像**

**物理图像:** 拉普拉斯最低本征向量 (低频模) 近似 “光滑夸克场分布”, 作为蒸馏基底; stout 平滑把规范场变光滑, 等价于剔除高动量 (紫外) 噪声, 使低频模更稳定。

**A6 任务如何正确执行.** `load(key) → stout_smear(nstep, rho) → 逐 t 片 calc(t)` (`test_eigenvector.py:44-50`)。

**A7 实际执行情况.** 本机无 QUDA (`libquda.so` 缺失) 且 LFS 数据未拉取, 本征求解路径未运行; `cupy` 设备可用 (1 GPU)。未验证。

**A8 实际输出情况.** 无数据相关输出; 本机仅验证 `import lattice` 与逻辑测试。

**A9 结果分析.** 求解残差/相位一致性测试 (`test_eigenvector.py:25-36`) 依赖参考数据 `.eigenvector.ref.npy` (LFS), 本机未产生。

**A10 综合分析.** 三路径共享同一 `calc` 接口 (`:356-370`), 工程上是 “后端无关” 设计; 物理上 `scipy/cupyx` 与 QUDA 求解的差别仅在性能与多项式加速 (`:295-299` 切比雪夫)。

**A11 结尾总结.** **核心算法:** (??) 的 `LinearOperator + eigsh(SA)` 三路径 + stout 平滑 (`NDArray/CUDA/QUDA`); GPU 数据路径本机未验证。

**A12 结尾评价.** 优点: 三后端路径与平滑三实现可切换; 缺点: CUDA 核仅实例化 `double` 模板 (`eigenvector.py:38-42` 的 TODO), `numpy/cupy` 平滑代码三份重复。

**A13 补充说明.** 本征求解的实际耗时/精度无基准数据; CUDA 核数值一致性未实测。

**A14 参考源.** `eigenvector.py:11-26, 234-257, 356-370, stout_smear.cu:160-245, test_eigenvector.py:25-`

**A15 附录.** 无 (GPU 路径未运行)。

**2.3 C3 ElementalGenerator 导数/动量因子构造 (elemental.py)****代码-物理映射**

**映射原则:** 每个关键代码符号必有物理对象, 双向可查。

| 代码符号                         | 物理对象                    | 物理含义与参考源                                                                                                            |
|------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>num_derivative</code>  | 导数组合数                   | $(3^{n+1} - 1)/2$ ( $n$ 阶全组合); <code>elemental.py:48</code>                                                         |
| <code>derivative(n)</code>   | 导数算符 $\tilde{\nabla}^n$ | 3 进制方向编码; <code>derivative.py:23-33</code>                                                                          |
| <code>derivative_list</code> | 导数组合集                   | 0 阶到 $n$ 阶全部组合; <code>elemental.py:49</code>                                                                        |
| <code>MomentumPhase</code>   | 平面波 $e^{ip \cdot x}$    | 动量投影相位 (含 cb2 棋盘化); <code>phase.py:41-53</code>                                                                     |
| <code>_nd</code>             | 协变位移                    | $V_f = U_\mu(x)V(x + \hat{\mu}) - U_\mu^\dagger(x - \hat{\mu})V(x - \hat{\mu})$ ; <code>elemental.py:279-288</code> |
| <code>pick_left/right</code> | 乘积规则展开                  | $\tilde{\nabla}(AB) = (\tilde{\nabla}A)B + A(\tilde{\nabla}B)$ 的左右分配; <code>elemental.py:309-321</code>             |

| 代码符号             | 物理对象   | 物理含义与参考源                                |
|------------------|--------|-----------------------------------------|
| stochastic_coeff | 随机蒸馏系数 | 稀释 (dilution) 块对角系数; elemental.py:71-95 |

表 4: C3 代码-物理双向映射 (数据来源: 源码逐符号核验)

**A1 目的与前期准备.** Elemental  $\Phi = V^\dagger \mathcal{O} V$  是算符  $\mathcal{O}$  在蒸馏空间的表示, 导数/动量使算符携带量子数 (角动量、动量)。前置: 平滑规范场 + 本征向量。

**A2 输入.** 规范场、本征向量、num\_nabla、momentum\_list、可选稀释 (elemental.py:16-26)。

**A3 输出.** [num\_deriv, num\_mom, Ne, Ne] 张量 (elemental.py:56), 即

$$\Phi_{jk}^{(n)}(p; t) = \sum_x e^{ip \cdot x} [\tilde{\nabla}^n V(x)]_{j,a}^* V(x)_{k,a} \quad (2)$$

**A4 整体框架.** calc(t) 三层循环: 本征向量装入 (:297-298) → 导数组合循环 (:299) → 动量循环 (:322-337)。

**A5 关键细节.** 乘积规则展开: 对  $n$  阶导数, 把  $2^n$  个子集分配为“右作用”/“左作用”, 系数  $(-1)^{\#右}$  (:309-321)。此写法替代注释掉的 num\_nabla\_right 组合写法 (:301-308), 等价但更紧凑。

```

1     for e in range(V.shape[0]):
2         V[e] = eigenvector[t, e]
3     for derivative_idx, derivative in enumerate(self.derivative_list):
4         VPV[derivative_idx] = 0
5         # for num_nabla_right in range(len(derivative) + 1):
6         #     coeff = (-1)**num_nabla_right * comb(len(derivative), num_nabla_right)
7         #     right = self._nD(V, U[:, t], derivative[:num_nabla_right])
8         #     left = self._nD(V, U[:, t], derivative[num_nabla_right:][::-1])
9         #     for momentum_idx, momentum in enumerate(self.momentum_list):
10            #         VPV[derivative_idx, momentum_idx] += contract(
11            #             "zyx,ezyxc,fzyxc->ef", coeff * momentum_phase.get(momentum), left.conj(), right
12            #         )
13     for pick_nabla in range(2 ** len(derivative)):
14         pick_right = []
15         pick_left = []
16         pick = pick_nabla
17         for direction in derivative:
18             if pick & 1:
19                 pick_right.append(direction)
20             else:
21                 pick_left.append(direction)
22             pick >>= 1
23         coeff = (-1) ** len(pick_right)
24         right = self._nD(V, U[:, t], pick_right)
25         left = self._nD(V, U[:, t], pick_left[::-1])
26         for momentum_idx, momentum in enumerate(self.momentum_list):
27             if self.stochastic_coeff is None:
28                 VPV[derivative_idx, momentum_idx] += contract(
29                     "zyx,ezyxc,fzyxc->ef",
30                     coeff * momentum_phase.get(momentum),
31                     left.conj(),
32                     right,
33                 )
34             else:

```